

Adversarial Systems

Luc Alexander Lüthi

March 3rd, 2026

Contents

1	Introduction	1
1.1	Offensive Security	1
1.2	Formalism	2
2	On Chess	2
3	On Argument	2
4	Signals, Observability, and Parsitism	3
5	Higher Order Strategy	3
5.1	Pure Substrate	4

1 Introduction

There are three kinds of job that relate to systems. Operators are tool runners, they get the job done but typically aren't as technical as their counterparts, even in roles which benefit from technicality. Engineers build the tools operators use. They identify classes of problems to solve and create infrastructure to allow operators to work effectively. Academics build the theory that underlies the systems engineers manipulate. This builds a pipeline, theory can take decades to make it into engineering and through to the field.

1.1 Offensive Security

I used to have a very distorted view of what a role in offensive security looked like. I thought offensive security specialists had to be top tier programmers. I thought people who operated at this level needed to be deeply technical because they were engineering solutions that worked against intended norms. The truth is however that while it is a relatively technical role, it's an operator role, not an engineering one. Many offensive security specialists are mediocre programmers, and it works out well for them. Penetration testing, red teaming, and exploit development are roles that evolved into operations from an engineering discipline it is only partially related to, cyber security is often associated with software engineering in circles outside of cyber security and software engineering. The role of security engineering is only just emerging to fill the role that the pipeline initially skipped over through a tangential engineering discipline.

Offensive security is not movie hacking. Movie hacking is not something that exists, nor did it ever. Our computing systems evolved out of academic formalisms which stressed correctness, so offensive computer hacking is a mostly rool heavy, often very tedious job. It is hard, but its a different hard from programming, which is what movie hacking often looks like. An engagement doesn't look like implementing a program that acts like a weapon and then letting it loose. Just like soldiers dont build canons on the field, penetration testers dont often write their own tooling, red teams don't usually hace an engineer

on payroll to write custom tooling. Beyond legal liability its too slow. You cannot reliably engineer a new custom exploit solution for a specific engagement, it's much more reliable to just use an existing toolset. These toolsets were engineered, but not by the people using them.

1.2 Formalism

This pulls at a question. Cyber security and physical security (really system security in general) are working backwards, they didnt start in an academic lab, they started out of fronline necessity. Engineering for this field emerged to sustain it. Will formalism follow?

It's hard to imagine what security formalism looks like. Security as math is simple, so simple you wouldn't think there is anything to talk about. A security engagement looks very different depending on which side you are standing on though. One side builds a system, secures it, tried to prevent and catch intrusion attempts. The other side tried to break in, leverage for control, sometimes the aim is destruction, sometimes the aim is parasitism, sometimes its intrusion itself. Once we see exploitable systems from a formalized perspective, we begin to understand the process of exploitation, and we begin to see what value formalism in security might have.

2 On Chess

Chess is actually a very good example of a game which encodes system security dynamics, from both the defensive and offensive sides. You build up a structure, claim it's secure, and try to poke holes in the structure of your opponent, finding intrusions into their position and escalating your power. Often the game is less about destruction and more about control. In chess, the security constraints are embedded in the structure. We see this pattern in some but not all exploitable systems. Pieces defend portions of structure, if you attack or attempt to damage the structure, the structure encodes potential reactivity. The ultimate arbiter of action is still the player however. This isn't a computed system, and chess moves are not self programmable, you need an external thinking or algorithmic entity to decide what the best move is.

In some systems, structure is inseparable from action. When you exploit a system you are not exploiting a substrate structure, you are exploiting the decision maker, the software, the immune system, the strategy of a player in a chess game. Programmed systems are sometimes thought to hold structure together with action, but in most systems which use computation to operate or secure, the structural substrate is actually removed from the deciding and acting component. Pure software systems are an exception to this. Lambda calculus and forth come to mind. In many foth-like languages, two things are true: every string of valid tokens produces a valid program, and every subset of a valid program is also a valid separate program. In theory, every input to a forth program could be reinterpreted as another forth program. These are systems which self metabolize. Chess is not like one of these systems. Structure in chess is not a pure substrate, but it does encode some of the decision process. You can tell bits and pieces about what your opponent is thinking by what their position looks like. This is enumeration, but distinctly its not enumeration about the substrate. Truthfully its never about the substrate, its about the actions and decisions made by the system. The system you are exploiting in chess is your opponent, but you play the game as though what you are exploiting is their position.

3 On Argument

Security poses a claim: this is secure, it cannot be exploited to perform action X without proper authorization, it cannot be used to do Y. It is the job of a defensive security operator to argue that a proof of that cliam exists. The role of an attacker is to provide a counter example to that proof. Chess encodes one of these claims. "My king cannot have its safety compromised". Your decision making is the proof, your opponents decision making encodes an attempt at a counter example.

You can encode this more directly as a proof writing game. One player makes claims toward some goal, provides evidence for some, but may not be able to provide evidence for all. This is where exploitability hides. The opponent is then tasked with showing that the univentional claims do not hold, providing counter examples in the unencodable holes of the system. Most exploits happen because some invariant is assumed by the design and is not enforced as a constraint, this game models that directly. Its the first model of exploitation dynamics as a game I made. It misses something though. The substrate here is arbitrary, you care only about the decisions and the strategy of the system you are exploiting. It's as pure a system of exploitation dynamics as you could get. This game tends to match the cadence of a verbal argument. Without a substrate however it doesn't provide very much use toward a formalism, and it's hard to encode this into something that we are able to reason about. It is ridiculously difficult to develop generic strategies for writing arbitrary proof on mixed or missing evidence that actually turns out to be useful in a research context. Because logic and programming encode truth and correctness so heavily, you get very little ambiguity. Any ambiguity present in a proof based system means the defending entity is immediately plainly aware of where their weaknesses are, which is not faithful to real world systems. Real world claims of security are often embedded in a combinatorial explosion of logical error, not in discrete ambiguity cases. This is the most obvious tool to reach for when it comes to a formalism for security, but it's the wrong tool. If you choose a substrate, like an arbitrary branch of math, it's actually a fun competitive sport.

4 Signals, Observability, and Parsitism

Exploitable systems operate on signals. They actually operate similar to language interpreters, and language interpreters and compilers are actually great candidate systems to stress test. They are signal translators that output a ton of information. Enumeration becomes natural in these kinds of systems. Signals are just any information you feed the system. These signals can be metabolized into action, sometimes this action changes the system, sometimes it reveals more information back to you. Oftentimes it will reveal information to some level of observability. Levels of observability are earned through gaining unintended capabilities. As you progress your capabilities in a system, you may encounter that what you can observe changes, and enumeration becomes a continuous process rather than a pre foothold activity.

Sometimes systems, especially computing systems, allow you to embed smaller machines of operation within them. Systems which allow this are a special class of system. Neither chess nor proofs allow for this level of exploitation. This is where malware and persistence live. Parasitism allows for complete control over a system, a full compromise. This is more detrimental than destruction and provides a higher classification of capability. This is not defeat, this is ownership.

5 Higher Order Strategy

There are a handful of reasons chess falls short as an offense/defense security sport. It's also important to note that I understand this isn't its intended goal either. For one, the environment is entirely static. This allows theory to become studying specific positions and memorizing lines. In the real world, this provides no analog that actually pans out to be an effective strategy of compromising a system. Beyond that, and slightly related, you are playing one instantiation of an adversary in chess. You are playing a member of a species when you play chess. Adversarial formalism must be concerned with classifications of security systems and adversarial systems. You may employ a strategy, and it may work against your opponent, but that does not mean it was a good strategy, that simply means your opponent was not sufficient. A good security system doesn't just deter the average opponent, it deters everyone. A good offensive security specialist doesn't just execute tool pipelines, they reason about what tool to use when.

In game theory we talk about games as mathematical objects. Entities attempt to find strategies that

may or may not exist to optimize a hypothetical function and win the game. It may be tempting to reach for this abstractions and say "that's adversarial systems, exploitations are strategies, security software is a strategy", but what we really need at the level of formalism is something which can operate over classifications of these things. If our base unit of understanding is a strategy itself, then reasoning over those strategies requires a higher order strategy. It requires us to study how strategies produce different invariants in adversarial systems.

5.1 Pure Substrate

Suppose the existence of a directed graph. It may be a hypergraph, it may be a meta hypergraph, it doesn't really matter. The simplest case that works is a simple directed graph. Structures present in this directed graph substrate present capabilities. Players are given a graph, as well as a presense structure. Anywhere this structure exists allows them to take actions. Control flow flows through the directed graph, nonrecursively. Any capability structure flow passes through allows an action. Players graphs turn into turn descriptions. Capability structures exist for growing your network, pruning your network, and normalizing portions of your network down to atomic structures to show the existence of a higher order emergent capability in your substrate structure. Normalization involves collapsing temporarily a structure $a \rightarrow b \rightarrow c$ down to $a \rightarrow c$. Constraints are expressed through netwitive patterns which present like uncontrolled growth or uncontrolled decay. You may mutate both your own structure but also the structure of other opponent graphs. Embedding your control structure in another entity allows you to take extra actions on their turn supposing their control flow passes through your nested graph machine.

This is a minimal example that provides all that is necessary for studying adversarial and defensive strategies. From this we can extrapolate policies, heuristics, and strategy combinators necessary to build complex adversarial behavior and study what classifications of system exist and how to defend them.